

씬 전환 구조 — 3계층 설계

전체 그림

전용 로딩 씬 유지(전환 메모리 피크 ↓) + 스킵 수명 + 던전 왕복 = 리셋 방식

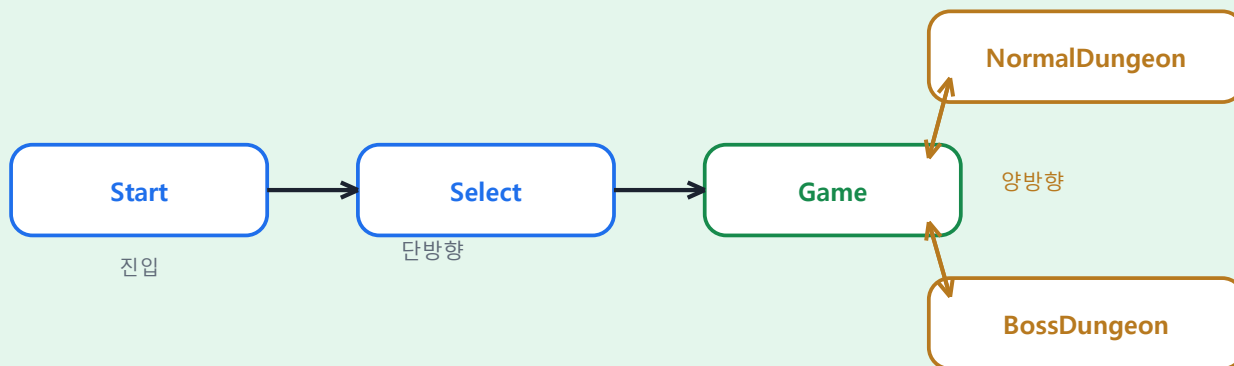
① 영속 계층 — @Managers (DontDestroyOnLoad)

모든 매니저(POCO) · SceneManagerEx가 전환을 지휘 · (선택) 페이드 오버레이 캔버스

② 전환 워커 — Loading 씬

Single 로드로 진입 → 이전 콘텐츠 씬이 완전 언로드되는 "빈 순간" 생성
(두 씬이 공존하지 않음 = 전환 피크 억제 핵심)

③ 콘텐츠 씬 (전부 BaseScene 파생)



핵심 원칙

1

지휘는 영속 계층

SceneManagerEx가 전환을 소유. 매 전환마다 파괴되는 씬 MonoBehaviour에 흐름을 두지 않음

2

Loading = 뷰 + 빈 순간

정책(무엇을 언제 해제/로드)을 갖지 않음. 진행률만.

3

해제는 딱 한 곳

스킵 회전(ChangeSceneScope) 한 지점에서만 리소스 해제

책임 분리 + 데이터 슬림화

컴포넌트별 역할

SceneManagerEx (POCO·영속)

전환 오케스트레이터 — 무엇을/언제 전환·해제·프리로드·활성화

LoadingScene (BaseScene)

트리거 + 자기 UI 참조 전달 (정책 없음)

LoadingSceneController

뷰 — 진행률/연출

SceneTableSO

순수 데이터 — enum → (씬 이름, preload 라벨)

ResourceManager

ChangeSceneScope() + 프리로드를 SceneScope로

컴포넌트

데이터: SceneDataSO 삭제 → 테이블로 통합

SceneDataSO (삭제 대상)

preloadLabels	유일하게 살아있음
sceneAddress	어디서도 안 읽음
clearPreviousMemory	안 읽음 (항상 clear)
sceneType	테이블 키와 중복

preloadLabels만 승격

SceneTableSO.Entry (통합)

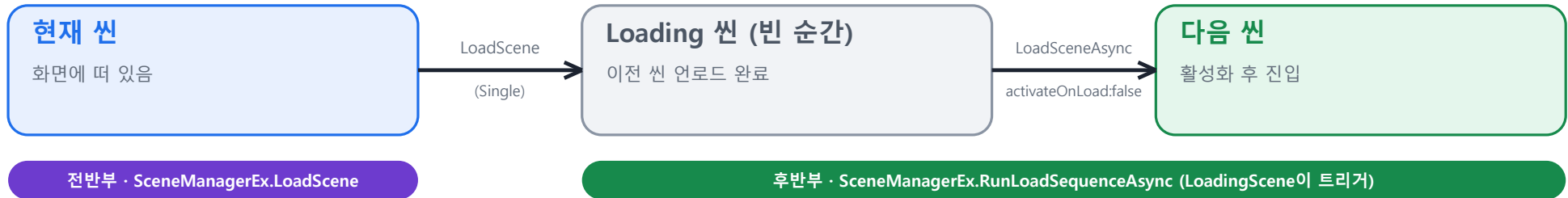
```
public Define.Scene sceneType;
public string    sceneName; // 비면 enum명
public string[]  preloadLabels;
```

SO 1개만 유지 · 에셋 12개 + 죽은필드 3개 제거

전환 시퀀스 — 3 씬 상태를 지나며

핵심 흐름

전용 로딩 씬이 "빈 순간"을 만들어 현재 씬을 완전히 비운 뒤 다음 씬을 로드 → 두 씬이 공존하지 않음



전반부 (요청 접수)

- ① 테이블 조회 → Pending 요청 생성
- ② 커버 ON
- ③ CurrentScene.Clear() — 씬 고유 정리만 (리소스 해제 X)
- ④ SceneManager.LoadScene("Loading")

후반부 (빈 순간 → 다음 씬 구축)

- ⑤ ChangeSceneScope() + Pool/UI Clear + UnloadUnused/GC
- ⑥ 프리로드 → 새 Scene 스코프로 (LoadDependenciesAsync)
- ⑦ LoadSceneAsync(다음, Single, 비활성) + 진행률
- ⑧ 연출 딜레이
- ⑨ ActivateAsync → 진입 → 커버 OFF

⑤ 단일 해제 지점 = ChangeSceneScope()

- 이전 Scene 스코프 Dispose → 옛 씬 에셋 회수
- 새 Scene 스코프 생성 (다음 프리로드가 여기 담김)
- 이전 씬은 이미 언로드된 뒤라 "렌더 중 해제 창" 없음 (P3)

현재 대비 바뀌는 것

- 전환 로직 LoadingScene → SceneManagerEx로 이동
- CurrentScene.Clear()의 리소스 해제 제거 (이중 Clear 소멸)
- NextSceneData(SO) → Pending 값 struct
- 프리로드가 SceneScope로 → 다음 회전 때 자동 회수

Before — 해제가 두 군데

① SceneManagerEx: CurrentScene.Clear()

이전 씬 렌더 중 리소스 해제 = 언로드 창

② LoadingScene: Resource.Clear() 또

중복 호출 + 옛 2버킷만 비움

이른 해제 + 이중 Clear

After — 한 곳

이전 씬 언로드 완료(빈 순간)

Loading 씬 진입 = 화면에 옛 씬 없음



ChangeSceneScope() 단 한 번

새 스코프 생성 → 이전 Dispose 순서 강제

스코프 채움 / 비움

Global — 부팅 시 1회, 앱 종료까지 상주

SceneTable · 플레이어 · 공통 UI (전환마다 안 건드림)

Scene — 전환마다 회전(Rotate)

회전 = 이전 스코프 Dispose(에셋 회수) → 새 스코프 생성

Scene A (Game)

Scene B (Dungeon)

Scene C (Game-새로)

Game 진입

던전 전환

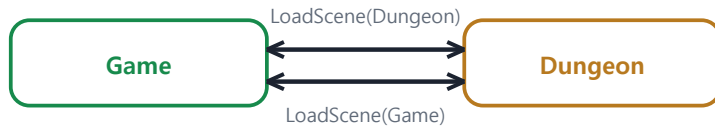
Game 복귀



던전은 별도 스코프 불필요

던전 = 씬이므로 Scene 스코프가 곧 던전 수명.
왕복 복귀 시 Game은 새 Scene 스코프로 재빌드
(상태 리셋 = 우리가 정한 방향).

양방향도 동일 파이프라인



- 복귀도 그냥 LoadScene(Game) → 같은 RunLoadSequenceAsync
- 특수 케이스 없음. 상태 보존이 필요해지면 그때 "허브 서스펜드"라는 다른 설계 (지금은 범위 밖)

지금 정할 것 2개

- 오케스트레이션 위치**
중앙화(SceneManagerEx) 권장 vs LoadingScene 유지+Clear만 교체(최소안)
- 씬 이름 방식**
테이블에 sceneName 명시 vs enum.ToString() 규약(필드 생략)

구현 순서 (각 = 독립 커밋)

- 데이터 슬림화** 동작 동일
SceneTableSO에 라벨 인라인 · SceneDataSO 삭제 · TryGet
- 흐름 이전** 동작 동일
RunLoadSequenceAsync를 SceneManagerEx로 · LoadingScene 축소 · Pending
- 스코프 배선** 동작 변화(P3+수명)
ChangeSceneScope로 교체 · BaseScene.Clear 해제 제거 · 프리로드 SceneScope
- 검증 → 커밋**
씬 왕복 후 잔존 핸들 = Baseline 확인